Specification: Strategies for the automated identification of service interfaces during the migration from monoliths to microservices

Alexander Ley
Computer Science Department
University of applied science Bonn-Rhein-Sieg
Sankt Augustin, Germany
s6alleyy@outlook.de

I. Context and Motivation

In Software Engineering of business applications, the architecture and organization of software components are playing a pivotal rule to enable desired features like *scalability*, *maintainability* and *deployability*.

As software become more and more complex these features are more and more less achieved due to the predominant implementation of *Monolithical Architectures* [1].

A. Monolithical Architectures

Monolithtical Architectures decompose the software according horizontal and technical oriented layers, which is illustrated in Fig. 1. Between these layers are often lots of dependencies resulting in a loss of flexibility whenever the software have to be adapted hindering *deployability* and *maintenance*. [2]

Monoliths are traditionally built as a single unit which can only be deployed and scaled as single unit. Therefore the deployability and scalability using a Monolithical Architecture is limited. [1] **< TODO: Letzten beiden Abschnitte anpassen >**

As the requirements towards the software are evolving there is need for software to be flexible, adaptable and deployable without influencing other parts of the software.

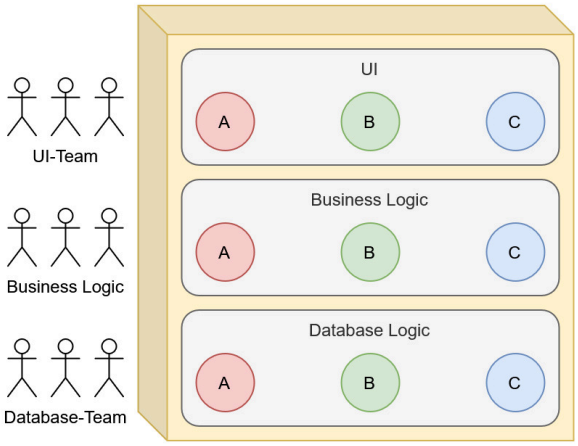

Fig. 1. Illustration of a business application built as a monolith. Original illustration from C. Nockemann [3].

B. Microservice Architecture

Due to the downsides of Monolithtical Architectures, researchers has reached out to other software paradigms [4].

Microservice Architectures seeks to overcome the shortcomings of monoliths and have therefore gained significant attraction [5]. The core idea behind this novel kind of software organization is to decompose the software vertically along enterprise business capabilities as illustrated in Fig. 2.

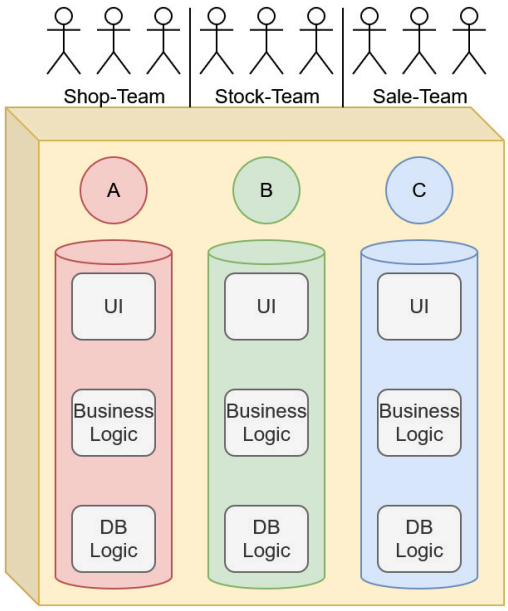

Fig. 2. Illustration of a business application built as microservices. Original illustration from C. Nockemann [3].

J. Lewis and M. Fowler [2] described the *Microservice Architecture* as follows:

> "The microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API [today often a RESTful API [5]]. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies."

Microservices are typically build in a way that they encapsulate one single business capability which can encompass:

1) User-Interface (UI) technology
2) Business logic
3) Database technology

Microservices seek to be as independent as possible by trying to minimize dependencies on other services. Therefore they can be maintained, adapted and deployed individually which satisfy more adequate companies desire for agile software development. **< TODO: Quelle? >**

NOTES:

C. Migration

- companies which have implemented monolithical architectures may consider to migrate to a Microservice Architecture
  - preliminary studies have shown that companies only have the source code as most up-to-date source of information about their legacy software systems available which may hinder forward-engineering strategies [6]
- existing monolithical applications have to become decomposed into smaller, independent services
- decomposing software into smaller parts have always been a challenge in software engineering and remains a complex and resource intensive task [7], [8]
- migration involves identifying service boundaries and packaging them into self-contained microservices with defined APIs [8]
- effective migration require robust strategies for deploying microservices and guaranteeing desired features like scalability, security and fault tolerance [8]

a) *Domain-Driven Design (DDD):*

- let migration engineer analyse and identify application's domain using techniques like *Domain-Driven Design (DDD)* to obtain service boundaries [1], [7]
- DDD is a collection of abstract concepts helping to model complex and large software with respect to the domain [3] **< TODO: In Seminararbeit DDD vielleicht genauer erklären und hier nur kurz >**

b) *Machine Learning (ML) [8]:*

- ML offer a promising avenue to tackle the manuel intervention and adaptability challenges
  - ML can further improve service boundary identification, analysing interdependencies and predicting failures
    - can assist in automating repetitive and error-prone tasks such as clustering related components or optimising deployment strategies
  - ML perform well in processing large and complex datasets and can uncover invisible patterns

⇒ can support decision-making processes which would be otherwise difficult to achieve with traditional methods

II. Probleme / Fragestellungen

Was genau ist Ihre Problemstellung? Welche Fragestelllungen behandeln Sie in Ihrer Arbeit?

III. Methodisches Vorgehen zur Zielerreichung

Wie gehen Sie bei der Erstellung der Ergebnisse vor (Arbeitsschritte) und welche Methodik praktizieren Sie dabei?

IV. Ziele und angestrebte Ergebnisse

Welche Ziele verfolgen Sie mit der Beantwortung der behandelten Fragen und welche wesentlichen Ergebnisse entstehen dabei?

V. Wissenschaftlicher Beitrag

Welchen Beitrag zur Bewältigung der Problemstellung leistet Ihre Arbeit? Welche neuen Erkenntnisse können Lesende Ihrer Arbeit erwarten?

References

[1] Y. Abgaz *et al.*, "Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, Aug. 2023, doi: 10.1109/TSE.2023.3287297.

[2] J. Lewis and M. Fowler, "Microservices: A Definition of This New Architectural Term." Accessed: Aug. 25, 2026. [Online]. Available: https://martinfowler.com/articles/microservices.html

[3] C. Nockemann, "Domain-Driven Design in Der Praxis," in *Presentations during the Lecture of Object-Oriented Component Architectures (OOKA) Renamed to Service-Oriented Component Architectures (SEKA)*, University of applied science Bonn-Rhein-Sieg, 2025.

[4] M. Daoud, A. E. Mezouari, N. Faci, D. Benslimane, Z. Maamar, and A. E. Fazziki, "Automatic Microservices Identification from a Set of Business Processes," in *Proceedings of the Third International Conference on Communications in Computer and Information Science (SADASC 2020)*, M. Hamlich, L. Bellatreche, A. Mondal, and C. Ordonez, Eds., Cham: Springer International Publishing, 2020, pp. 299–315. doi: 10.1007/978-3-030-45183-7_23.

[5] M. Garriga, "Towards a Taxonomy of Microservices Architectures," in *Proceedings of the 15th International Conference on Software Engineering and Formal Methods (SEFM 2017)*, A. Cerone and M. Roveri, Eds., Cham: Springer International Publishing, 2018, pp. 203–218. doi: 10.1007/978-3-319-74781-1_15.

[6] M. Abdellatif *et al.*, "A Taxonomy of Service Identification Approaches for Legacy Software Systems Modernization," *Journal of Systems and Software*, vol. 173, p. 110868, Mar. 2021, doi: 10.1016/j.jss.2020.110868.

[7] M. Gysel, L. Kölbener, W. Giersche, and O. Zimmermann, "Service Cutter: A Systematic Approach to Service Decomposition," in *Proceedings of the 5th European Conference on Service-Oriented and Cloud Computing (ESOCC 2016)*, M. Aiello, E. B. Johnsen, S. Dustdar, and I. Georgievski, Eds., Cham: Springer International Publishing, 2016, pp. 185–200. doi: 10.1007/978-3-319-44482-6_12.

[8] I. Trabelsi, B. Mahmoudi, J. B. Minani, N. Moha, and Y.-G. Guéhéneuc, "A Systematic Literature Review of Machine Learning Approaches for Migrating Monolithic Systems to Microservices," *IEEE Transactions on Software Engineering*, vol. 51, no. 11, pp. 2972–2995, Nov. 2025, doi: 10.1109/TSE.2025.3603897.